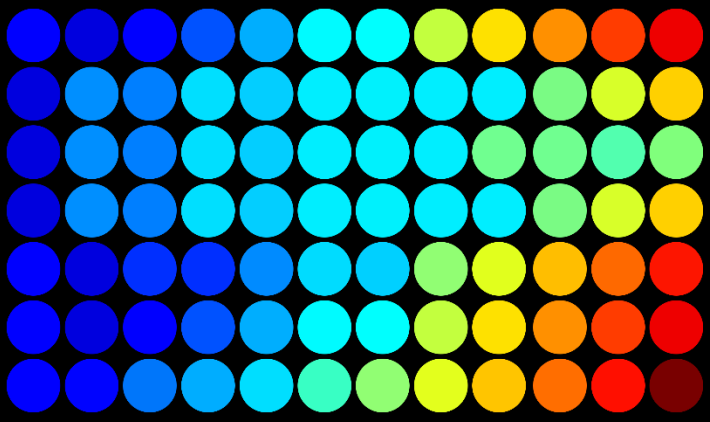


STANLY FEA 2017

FRAME SOLVER

Copyright © Tommaso Cavandoli

MATLAB® R2017b

INPUT MANUAL

Index

General info 2
- Structure and syntax 2
- Measure units 2

Static Analysis (`.stan` file) 3
- Mesh tables 3
  - Nodes (mandatory) 3
  - Elements (mandatory) 4
  - Spring elements 4
  - Mesh control 4
- Property tables 5
  - Section properties (mandatory) 5
  - Material properties (mandatory) 5
  - Stress concentration factors 6
- Constraints tables 6
  - Ground constraints (mandatory) 6
  - Internal constraints 7
  - Yielded constraints 8
- Loads tables 9
  - Nodal loads 9
  - Distributed loads 9
  - Thermal loads 10
  - Gravity effect 10
- Table priorities 11

Fatigue Analysis (`.fstan` file) 12
- Property tables 12
  - Section properties (mandatory) 12
  - Material properties (mandatory) 12
  - Stress concentration factors 13
- Alternate and mean loads 13

General info

This manual will explain how to create the input file for STANLY. There are two kinds of file:

- STAN for static analysis
- FSTAN for fatigue analysis

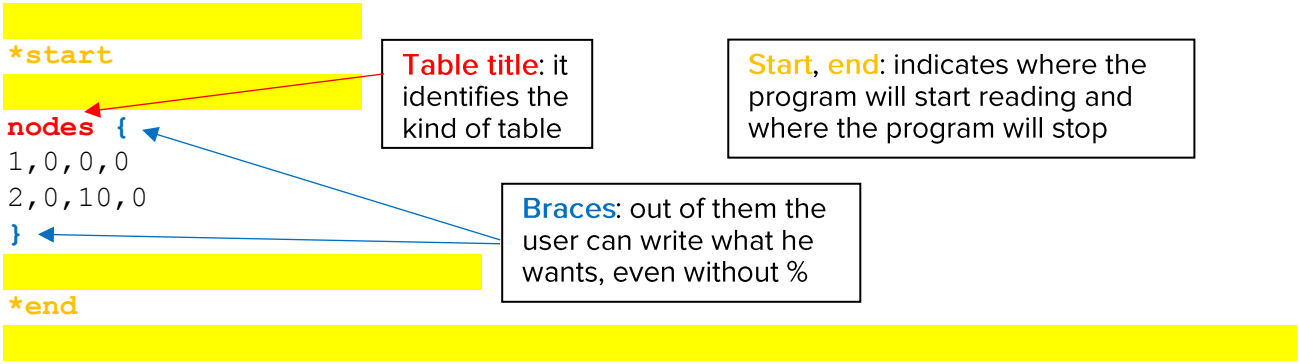
While STANLY is running is possible to open a blank model for each file type *(File > Blank model)*. Once it's open it must "Save As" in other destination, or any modification won't be saved. These files must be edited with `notpad.exe` or inside `matlab.exe` editor.

Structure and syntax

Anything written before `*start` or after `*end` will never be read. To remark something, you can use `%` also as in-line command. For instance:

```
*start
nodes { %now start acquiring nodes
% n x y z
1 0 0 0 % This is the first node
2 0 10 0 % This is the second one
} %this brace is important because communicates that the node tab is
%finished
```

Braces are an important feature: every tab must be written inside the space between `*start` and `*end`. In the first line, it must have the table name and an open graffiti brace, then the table data, and finally a closed brace in a new-line:



Every information written in the yellow side will be neglected. Data inside the braces can be divided by spaces or commas (as MatLab® array building)

Measure units

The measure units are automatically user defined. For instance, if the nodes coordinates are indicated in *millimeters*, and forces in *newton*, stress/pressure unit will be *megapascal*. Fundamental units used are the following:

- length unit **[lu]**
- force unit **[fu]**
- temperature unit **[tu]**

From them we can derives all the other units

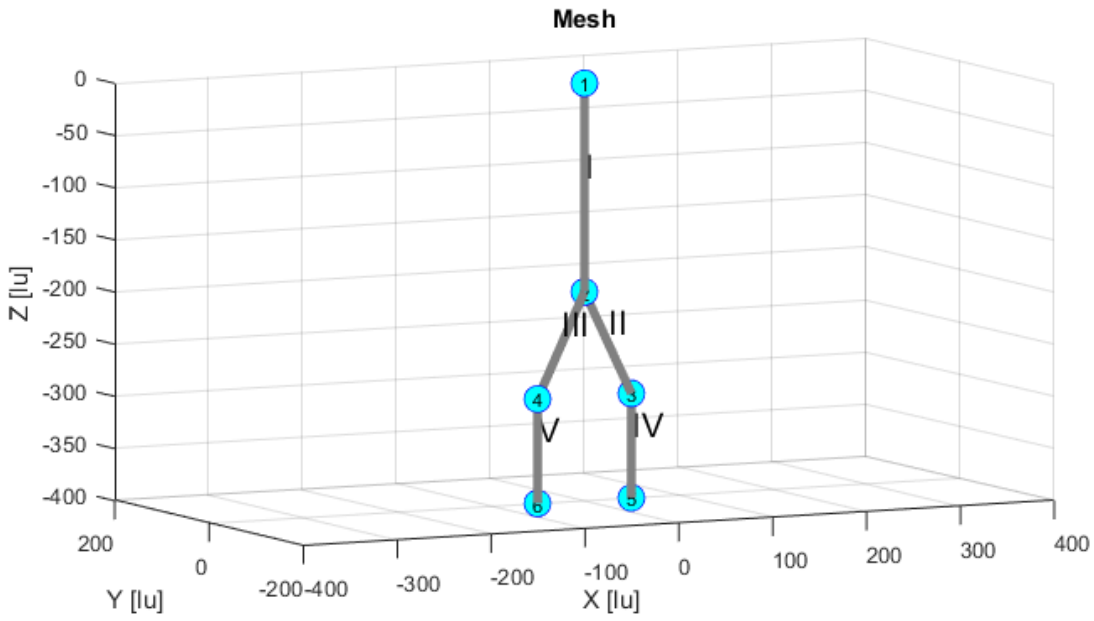
Static Analysis (`.stan` file)



This extension is made just for static analysis. It has ***14*** possible tables, ***5*** of these are mandatory to successfully load the file. The optional tables can be left empty or even excluded by the paper. The icon associated is figured besides.

Mesh tables

The following tables will define the mesh of the model. Below `bicycle.stan` file's mesh is illustrated:



Nodes (mandatory)

Every row corresponds to a node. Columns are described below:

n	Identification numbers must start from `1` to node number: no node can have the same, and there can't be any number left.
x	Position along X in [lu]
y	Position along Y in [lu]
z	Position along Z in [lu]

Example: `bicycle.stan`

```
nodes  {
%n          x           y           z
1           0           0           0
2           0           0           -200
3           50          0           -300
4           -50         0           -300
5           50          0           -400
6           -50         0           -400
}
```

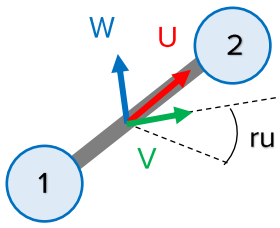
Elements (mandatory)

Every row corresponds to an element. Columns are described below:

e	Identification numbers must start from `1` to elements number: no element can have the same, and there can't be any number left.
n1	ID number of the first node of the element
n2	ID number of the second node of the element
s	ID number of the section of the element (from section matrix)
m	ID number of the material of the element (from material matrix)
ru	counter-clockwise rotation angle of the local reference system (U,V,W) around U, in [deg]. See *Output manual* to learn how elements orientation is computed.

Example: `bicycle.stan`

```
elements {
%e    n1    n2    s     m     ru
1     1     2     1     1     45
2     2     3     1     1     0
3     2     4     1     1     0
4     3     5     1     1     0
5     4     6     1     1     0
}
```



The orientation of the elements depends from the choice of node1 and node2. The local coordinates are the ones defined in the figure above (U, V, W).

Spring elements

They're element that has just axial stiffness. Stress and strain evaluation through this element aren't considered in results. Every row corresponds to a spring element. Columns are described below:

e	Identification numbers must start from `1` to springs number: no spring can have the same, and there can't be any number left.
n1	ID number of the first node of the element
n2	ID number of the second node of the element
K	Axial stiffness of the spring in [fu]/[lu]

```
springs {
%e          n1          n2          K
1           5           6           1e6
}
```

Mesh control

This command is made to make the mesh sharper along some user-chosen elements.

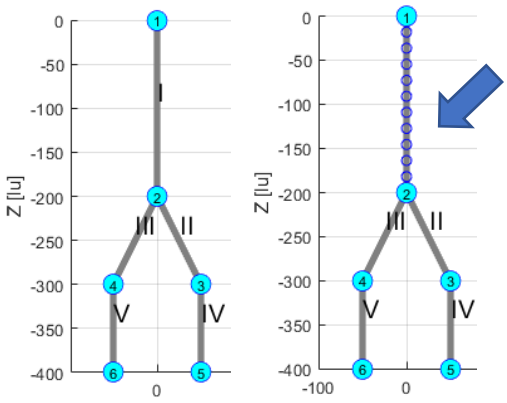
Caveat: imposing a mesh control on a loaded element (*see "Distributed loads"*), it may not have any effect.

e	Identification number of the mesh-controlled element
in	Number of new nodes to creates along element between node1 and node2 (if `0` will be automatically controlled)

Example: `bicycle.stan`

```
mesh control {
%e          in
1           10
}
```

In `bicycle.stan` there's no mesh control, so (as this table is not mandatory) It must be left empty or excluded from the paper. Here it's how the mesh changes introducing a mesh control as the one seen above (from left: with and without).



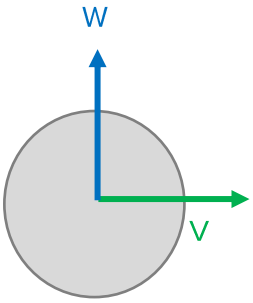
Property tables

The following tables will define the mechanical properties of the structure.

Section properties (mandatory)

Every row corresponds to a cross section. Columns are described below:

id	Identification numbers must start from `1` to sections number: no section can have the same, and there can't be any number left.
A	Cross section area [lu]^2
Jv	Cross section inertia around V axis in [lu]^4
Jw	Cross section inertia around W axis in [lu]^4
Wv	Cross section resistance modulus around V axis in [lu]^3
Ww	Cross section resistance modulus around W axis in [lu]^3
Tv	Shear factor along v (`0` for slender Euler-Bernoulli beams)
Tw	Shear factor along w (`0` for slender Euler-Bernoulli beams)



Example: `bicycle.stan`

```
sections {
%id         A           Jv          Jz          Wv          Wz          Tv    Tw
1           1.9635e+3   3.0680e+5   3.0680e+5   1.2272e+4   1.2272e+4   0     0
}
```

Material properties (mandatory)

Every row corresponds to a material. Columns are described below:

id	Identification numbers must start from `1` to materials number: no material can have the same, and there can't be any number left.
E	Young modulus in [fu]/[lu]^2
v	Poisson's coefficient
a	Thermal linear expansion coefficient in [tu]^ (-1)
m	Mass per unit length in [fu] * [sec]^2
YS	Yielding strength in [fu]/[lu]^2
UTS	Ultimate tensile strength in [fu]/[lu]^2

Example: `bicycle.stan`

```
materials {
%id         E           v           a           m           YS          UTS
1           2e5         .3          11e-6       1           500         700
}
```

Stress concentration factors

Every row is a user-chosen node with specified stress concentration factors. Here we're placing the notches of the structure. For the other nodes, the default value will be used.

n	Identification number of the node with notch
Kta	Axial load stress concentration factor (default: `1`)
Ktb	Bending stress concentration factor (default: `1`)
Ktt	Torque stress concentration factor (default: `1`)

```
notches {
%n          Kta         Ktb         Ktt
3           1           2           1.6
}
```

Constraints tables

The following tables will define infinitely rigid, elastic, binding cuts, inclined constraint for the structure. In this part degrees of freedom and degrees of boundary will be determined.

Caveat: For a good analysis, the structure must be at least iso-static and not fleeting. Otherwise a warning message will be displayed:

```
Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate.
```

Ground constraints (mandatory)

Every row corresponds to a constrained node. Columns are described below. The values of stiffness can be from `0` (free) to `inf` (infinitely rigid).

n	Identification number of the constrained node
Kx	Translational stiffness along X in [fu]/[lu]
Ky	Translational stiffness along Y in [fu]/[lu]
Kz	Translational stiffness along Z in [fu]/[lu]
Krx	Rotational stiffness around X in [fu]*[lu]/[lu]
Kry	Rotational stiffness around Y in [fu]*[lu]/[lu]
Krz	Rotational stiffness around Z in [fu]*[lu]/[lu]
th	Azimuthal inclination angle (around Z) in [deg]
ph	Elevation inclination angle (around Y) in [deg]

Example: `bicycle.stan`

```
ground constraints {
%node       Kx    Ky    Kz    Krx   Kry   Krz   th    ph
1           inf   inf   0     inf   inf   inf   0     0
5           inf   inf   1e6   inf   inf   inf   0     0
6           inf   inf   1e6   inf   inf   inf   0     0
}
```

Internal constraints

To model internal constraints the user must follow the instructions below. For this section, the files `beam_without.stan` and `beam_with.stan` will be used as instance:

1. *Duplicates the node in which the internal constraint must be placed. The original node will be called "master" and the other "slave":*

`beam_without.stan`

```
nodes {
%nod  x     y     z
1     0     0     0
2     .5    0     0
3     1     0     0
}
```

`beam_with.stan`

```
nodes {
%nod  x     y     z
1     0     0     0
2     .5    0     0 %master
3     1     0     0
4     .5    0     0 %slave
}
```

2. *Separate one of the two elements connected to the master node using the slave node instead:*

`beam_without.stan`

```
elements {
%el   n1    n2    sect  mat
1     1     2     1     1
2     2     3     1     1
}
```

`beam_with.stan`

```
elements {
%el   n1    n2    sect  mat
1     1     2     1     1
2     4     3     1     1
}
```

3. *Now the two elements are disjointed each other. Use the* `internal constraints` *table to connect just the internal constraint degrees of freedom of master and slave. This table must be used as following:*

m	Identification number of the master node
s	Identification number of the slave node
Ky	Translational stiffness along Y in [fu]/[lu]
Kz	Translational stiffness along Z in [fu]/[lu]
Krx	Rotational stiffness around X in [fu]*[lu]/[lu]
Kry	Rotational stiffness around Y in [fu]*[lu]/[lu]
Krz	Rotational stiffness around Z in [fu]*[lu]/[lu]

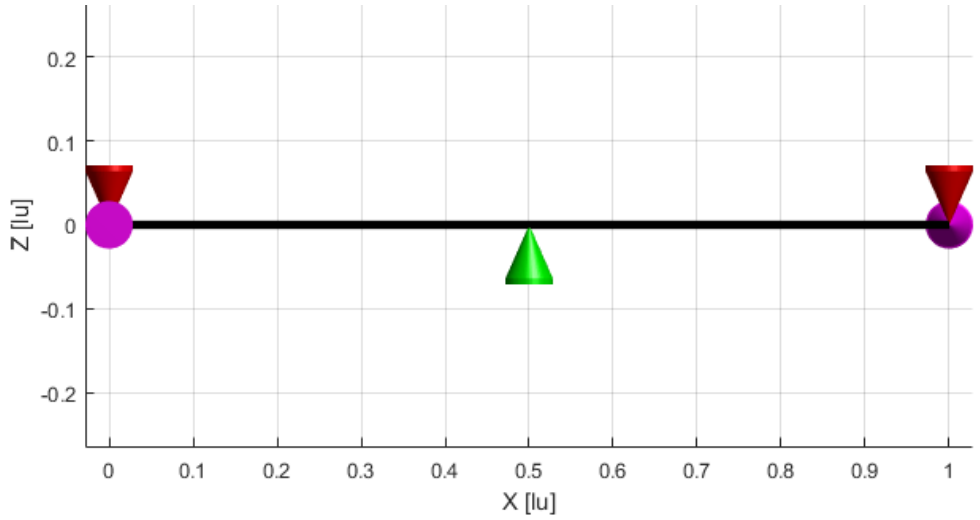
beam_without.stan

```
internal constraints {
%m s Kx  Ky  Kz  Krx Kry  Krz
}
```

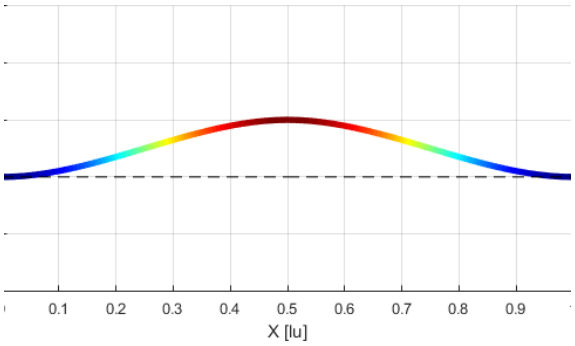
beam_with.stan

```
internal constraints {
%m s Kx  Ky  Kz  Krx Kry  Krz
2  4 inf inf inf inf 0    inf
}
```

4. *For a load and boundaries configuration as in figure, the two structures will behave differently:*

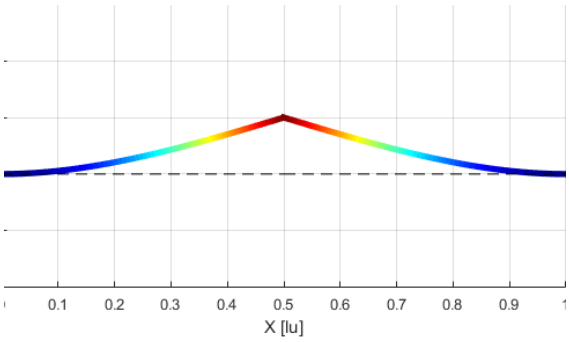


beam_without.stan



Continuous beam

beam_with.stan



Two beams connected by a pin

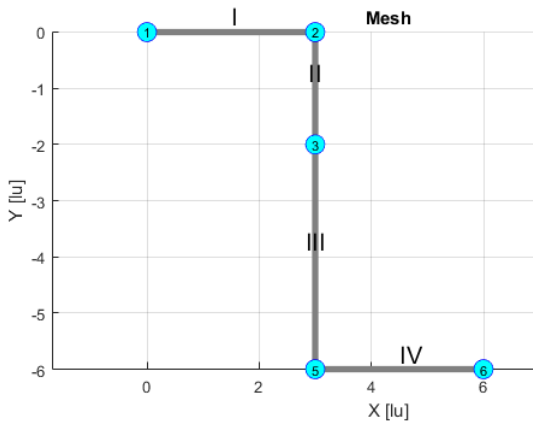
Yielded constraints

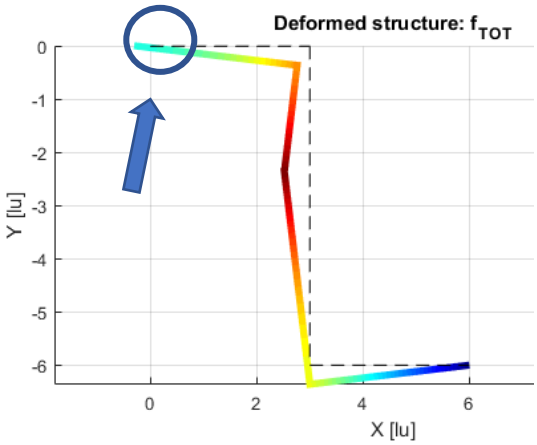
Every row corresponds to a binding cut node. Columns are described below.

n	Identification number of the yielded constraint node
x	Yielding along X in [lu]
y	Yielding along Y in [lu]
z	Yielding along Z in [lu]
rx	Yielding around X in [lu]/[lu]
ry	Yielding around Y in [lu]/[lu]
rz	Yielding around Z in [lu]/[lu]

Example: `binding_cut.stan`

```
yielded constraints {
%n     x      y      z      rx     ry     rz
1      -.01   0      0      0      0      0
}
```





Loads tables

The following tables will define every kind of external excitation.

Nodal loads

Every row corresponds to a node loaded. Columns are described below.

n	Identification number of the loaded node
Fx	Force along X in [fu]
Fy	Force along Y in [fu]
Fz	Force along Z in [fu]
Mx	Moment around X in [fu]*[lu]
My	Moment around Y in [fu]*[lu]
Mz	Moment around Z in [fu]*[lu]

Example: `bicycle.stan`

```
nodal loads {
%n     Fx     Fy     Fz     Mx     My     Mz
1      0      0      -7e5   0      0      0
}
```

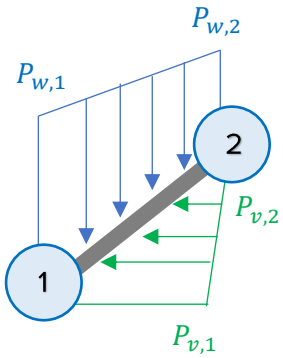
Distributed loads

Every row corresponds to an element loaded. Columns are described below. As the shape function used for beam elements doesn't match properly with distributed loads, the mesh in the loaded zone will be automatically made sharper.

Caveat: Every mesh control previously applied on the element at issue, will be neglected.

Transversal loads can be constantly or linearly distributed. For this reason, it's possible to set two values to describe them: one for the first node and the other one for the second.

e	Identification number of the loaded element
Pu	Force per unit length along U (local X) in [fu]/[lu]
Pv1	Force per unit length along V (local Y) in [fu]/[lu] in node 1
Pv1	Force per unit length along V (local Y) in [fu]/[lu] in node 2
Pw1	Force per unit length along W (local Z) in [fu]/[lu] in node 1
Pw2	Force per unit length along W (local Z) in [fu]/[lu] in node 2
Mqu	Torque per unit length around U (local X) in [fu]*[lu]/[lu]
in	Number of new nodes to creates along element between node1 and node2 (if `0` will be automatically controlled)



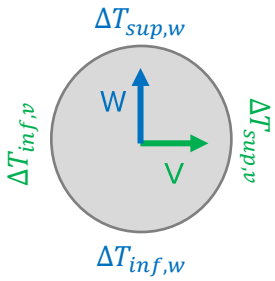

```
distributed loads {
%e    Pu    Pv1   Pv2   Pw1   Pw2   Mqu   in
1     1     0     0     -2    1     0     0
}
```

Caveat: assign a distributed load as: `[1 0 0 0 0 0 0 0]` is equal to put an automatic mesh control on the element 1.

Thermal loads

Every row corresponds to an element loaded. Columns are described below.

e	Identification number of the loaded element
Dt_sup_v	ΔT on positive V (local Y) cross section border in [tu]
Dt_inf_v	ΔT on negative V (local Y) cross section border in [tu]
Dt_sup_w	ΔT on positive W (local Z) cross section border in [tu]
Dt_inf_w	ΔT on negative W (local Z) cross section border in [tu]




```
thermal variations {
%e    Dt_sup_v    Dt_inf_v    Dt_sup_w    Dt_inf_w
1     .3          -.2         0           0
}
```

Gravity effect

One single row. Columns are described below. A mesh control will be automatically applied for all the not mesh-controlled elements.

Caveat: To manually mesh control see the section *Mesh control* or *Distributed loads*.

ax	Gravity acceleration along X
ay	Gravity acceleration along Y
az	Gravity acceleration along Z

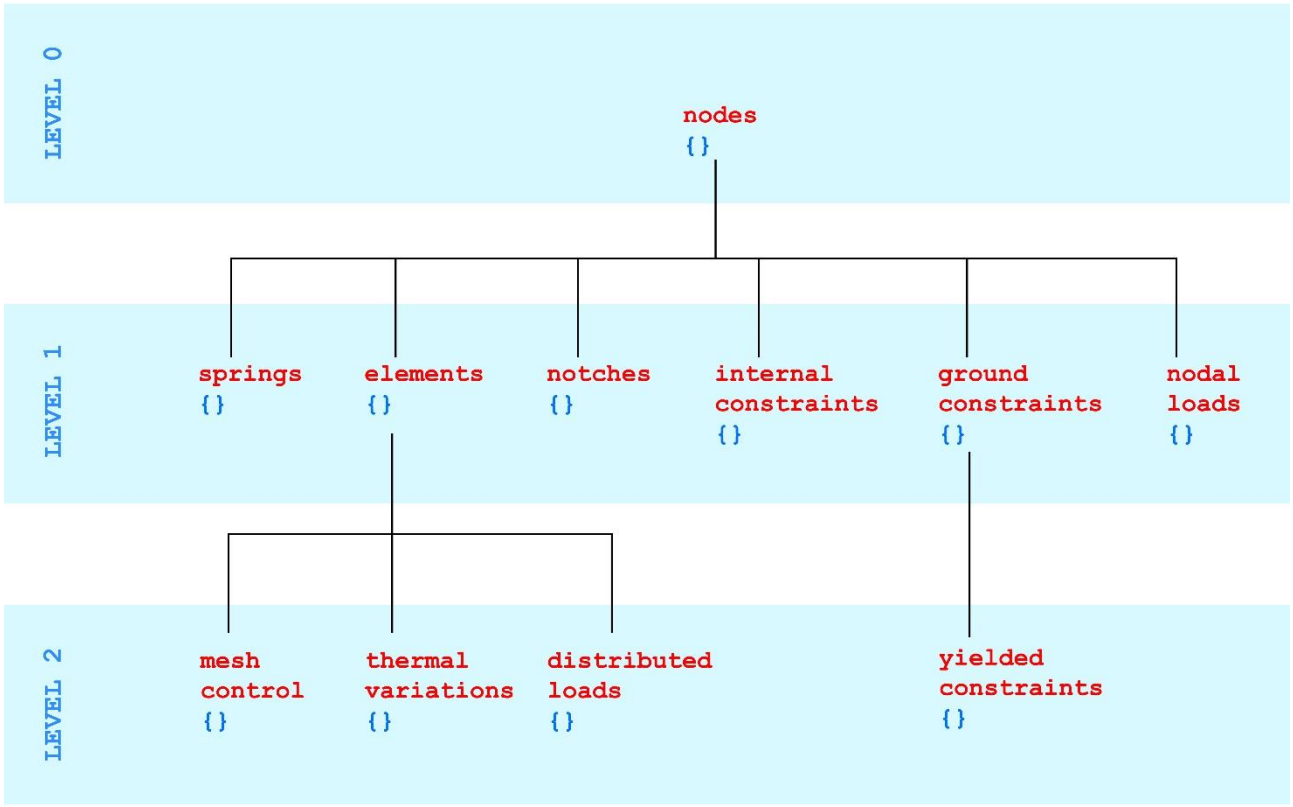
```
gravity {
%ax   ay    az
0     0     -9.81
}
```

Caveat: priority assignment for mesh controls is:

1. `distributed loads {}`
2. `mesh control {}`
3. `gravity {}`

Table priorities

Sometimes, to define some table, other ones are needed. For this reason, the user must follow a precise order to declare tables. In the following tree chart, the priority scheme is illustrated. The arrows go from *Parent* to *Child* table. The ones that don't appear in the chart can be randomly ordered.



Fatigue Analysis (`.fstan` file)



This extension is made for fatigue analysis. The tables are exactly the same then before, exception made for *Property tables*. The icon associated is figured besides.

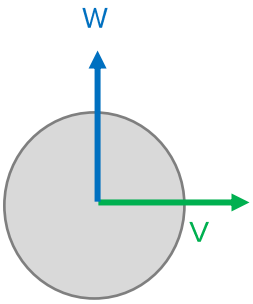
Property tables

The following tables will define the mechanical properties of the structure.

Section properties (mandatory)

Every row corresponds to a cross section. Columns are described below:

id	Identification numbers must start from `1` to sections number: no section can have the same, and there can't be any number left.
A	Cross section area [lu]^2
Jv	Cross section inertia around V axis in [lu]^4
Jw	Cross section inertia around W axis in [lu]^4
Wv	Cross section resistance modulus around V axis in [lu]^3
Ww	Cross section resistance modulus around W axis in [lu]^3
Tv	Shear factor along v (`0` for slender Euler-Bernoulli beams)
Tw	Shear factor along w (`0` for slender Euler-Bernoulli beams)
md	Dimension fatigue factor



Example: `bicycle.fstan`

```
sections {
%id   A           Jv          Jz          Wv          Wz          Tv    Tw    md
1     1.9635e+3   3.0680e+5   3.0680e+5   1.2272e+4   1.2272e+4   0     0     .7
}
```

Material properties (mandatory)

Every row corresponds to a material. Columns are described below:

id	Identification numbers must start from `1` to materials number: no material can have the same, and there can't be any number left.
E	Young modulus in [fu]/[lu]^2
v	Poisson's coefficient
a	Thermal linear expansion coefficient in [tu]^(-1)
m	Mass per unit length in [fu] * [sec]^2
YS	Yielding strength in [fu]/[lu]^2
UTS	Ultimate tensile strength in [fu]/[lu]^2
ms	Surface fatigue factor

Example: `bicycle.fstan`

```
materials {
%id         E           v           a           m     YS    UTS   ms
1           2e5         .3          11e-6       1     500   700   .95
}
```

Stress concentration factors

Every row is a user-chosen node with specified stress concentration factors. Here we're placing the notches of the structure. For the other nodes, the default value will be used.

n	Identification number of the node with notch
Ktb	Bending stress concentration factor (default: 1)
Ktt	Torque stress concentration factor (default: 1)
q	Notch sensitivity (default: 1)

```
notches {
%n          Ktb         Ktt         q
3           2           1.8         .9
}
```

Caveat: In fatigue analysis (unlike static one), the notch information will be used to compute stress limits:

$$\sigma_{FA,lim} = \frac{0{,}5 \cdot \sigma_{UTS} \cdot m_s \cdot m_d}{1 + q \cdot (K_{t,b} - 1)} \qquad \tau_{FA,lim} = \frac{0{,}28 \cdot \sigma_{UTS} \cdot m_s \cdot m_d}{1 + q \cdot (K_{t,t} - 1)}$$

Sines criterion will use just $\sigma_{FA,lim}$.

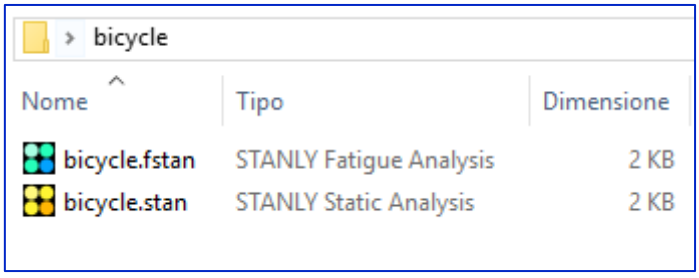
Alternate and mean loads

In a `.fstan` file, every load is considered as completely reversed load. If also mean loads must be considered:

1. *Save the current file as "`myfile.fstan`" including only the completely reversed loads.*
2. *Save another file as "`myfile.stan`" with same name, in same directory*
3. *The following tables in the two files must be the same:*

   `nodes {}, elements {}, springs {}, mesh control {}, sections {}, materials {}, ground constraints {}, internal constraints {}.`

4. *Include in "`myfile.stan`" just the static loads.*
5. *Running the `.fstan` file, also the static effects described in `.stan` file will be considered.*



All the examples are in `.\structures`